

DevSecOps Pipeline Report

TP1 · WeatherOps · SecNotes · CRUD App · AWS Terraform

Rapport Final

Chaîne DevSecOps Complète

Travaux Pratiques DevSecOps — Synthèse Globale

Node.js · Express · MongoDB · Docker · Kubernetes · Terraform
GitLab CI/CD · GitHub Actions · SonarQube · OWASP ZAP · AWS EC2/EKS
Jest · Supertest · Trivy · Secret Detection · Shift Left Security

Auteure

Asma Daoui

4^{ème} année – Cybersécurité

 github.com/BeeHope

Module

DevSecOps

Année académique 2024–2025

 Juin 2026

Résumé Exécutif

Synthèse du Parcours DevSecOps

Ce rapport de synthèse présente l'ensemble du parcours DevSecOps réalisé au cours du module, à travers **5 projets progressifs**. Chaque projet a apporté de nouvelles compétences : de la première conteneurisation Docker (TP1 Notes App) jusqu'au déploiement d'un cluster EKS AWS via Terraform (SecNotes), en passant par la mise en place de pipelines CI/CD complets avec tests automatisés, analyses SAST/DAST, et orchestration Kubernetes.

Technologies maîtrisées : Node.js, Express, MongoDB, Docker, Kubernetes, GitLab CI/CD, GitHub Actions, SonarQube, OWASP ZAP, Trivy, Terraform, AWS EC2/EKS, Jest, Supertest.

Projet	Application	Pipeline	Spécificité
TP1	Notes App	GitHub Actions	Première CI/CD
TP2	WeatherOps	GitLab CI/CD	Kubernetes + AWS EC2
TP3	SecNotes	GitLab CI/CD	SAST SonarQube + DAST ZAP
TP4	CRUD App	GitLab CI/CD	Trivy + Docker Desktop K8s
TP5	Terraform AWS	GitLab CI/CD	Infrastructure as Code EKS

Contents

Résumé Exécutif	1
1 Introduction et Vision DevSecOps	4
1.1 Qu'est-ce que le DevSecOps ?	4
1.2 Les 3 Piliers du DevSecOps	4
1.3 Architecture DevSecOps Globale du Parcours	4
2 TP1 — Notes App : Premiers Pas DevSecOps	6
2.1 Présentation du Projet	6
2.2 Architecture de l'Application	6
2.3 Dockerfile	6
2.4 Tests Automatisés — Jest	7
2.5 Pipeline CI/CD — GitHub Actions	7
2.6 Résultats de l'Exécution Docker	8
3 TP2 — WeatherOps : Pipeline DevSecOps Complet	9
3.1 Présentation du Projet	9
3.2 Architecture du Projet	9
3.3 Endpoints REST	10
3.4 Pipeline GitLab CI/CD — 5 Stages	10
3.4.1 Stage 1 — Test (Jest/Supertest)	10
3.4.2 Stage 2 — Security (Détection de Secrets)	10
3.4.3 Stage 3 — Build (Docker)	11
3.4.4 Stages 4 & 5 — Deploy Staging et Production	12
3.5 Infrastructure AWS — Terraform	12
3.6 Pipeline Réussi	12
4 TP3 — SecNotes : SAST, DAST et Kubernetes	14
4.1 Présentation du Projet	14
4.2 Vulnérabilités Pédagogiques	14
4.3 Pipeline GitLab CI/CD — 7 Stages, 12 Jobs	14
4.3.1 Stage SAST — SonarQube	15
4.3.2 Stage DAST — OWASP ZAP	15
4.3.3 Infrastructure as Code — Terraform EKS AWS	15
4.4 Résultats — Pipeline #2435952558	16
5 TP4 — CRUD App : Trivy et Kubernetes Multi-Replicas	17
5.1 Présentation du Projet	17
5.2 Dockerfile Multi-Stage	17
5.3 Pipeline GitLab CI/CD — 4 Stages	18

5.4	Kubernetes — 3 Replicas + StatefulSet MongoDB	18
6	TP5 — AWS Terraform : Infrastructure as Code	20
6.1	Présentation du Projet	20
6.2	Configuration IAM AWS	20
6.3	Configuration Terraform	21
6.4	Pipeline GitLab CI/CD — Stages Package et Deploy	21
6.5	Résultats	22
7	Analyse de Sécurité Transversale	23
7.1	Outils de Sécurité Utilisés	23
7.2	Matrice des Vulnérabilités par Projet	23
7.3	Bonnes Pratiques Appliquées	23
7.4	Améliorations Recommandées	24
8	Comparaison des Pipelines et Outils	25
8.1	Tableau Comparatif des Pipelines	25
8.2	GitLab CI/CD vs GitHub Actions	25
9	Conclusion et Bilan	26
9.1	Acquis du Module DevSecOps	26
9.2	Bilan des Livrables	26
A	Variables CI/CD GitLab — Référence Complète	28
B	Données Météo — WeatherOps	29
C	Commandes de Référence	30

1. Introduction et Vision DevSecOps

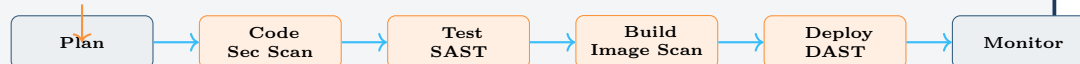
1.1 Qu'est-ce que le DevSecOps ?

Le DevSecOps est une évolution du mouvement DevOps qui intègre la sécurité (*Security*) comme une responsabilité partagée, continue et automatisée au sein du cycle de vie logiciel. Contrairement à l'approche traditionnelle où la sécurité était traitée en fin de cycle (souvent par une équipe dédiée), le DevSecOps positionne la sécurité **dès la première ligne de code**.

Le Principe "Shift Left Security"

"Shift Left" signifie déplacer les contrôles de sécurité vers la *gauche* du pipeline de développement, c'est-à-dire vers les phases les plus précoces possible.

Sécurité intégrée à chaque étape



1.2 Les 3 Piliers du DevSecOps

Pilier	Définition	Outils utilisés
Dev (Développement)	Écriture du code, tests unitaires, revue de code	Node.js, Jest, Supertest, VS Code
Sec (Sécurité)	Scan statique, détection de secrets, scan d'image, tests dynamiques	SonarQube, Trivy, OWASP ZAP, grep
Ops (Opérations)	Conteneurisation, orchestration, déploiement, IaC	Docker, Kubernetes, Terraform, AWS

1.3 Architecture DevSecOps Globale du Parcours

Vue d'Ensemble de l'Architecture

DÉVELOPPEUR (VS Code)
Code → Git Commit → Push (GitLab/GitHub)

Webhook / Push Event

PIPELINE CI/CD (GitLab / GitHub Actions)

```
[1] install → [2] test → [3] SAST → [4] build

      npm ci      Jest      SonarQube      Docker
                  Supertest   Trivy          Image
                                           Build/Push

[5] DAST ← [6] deploy
OWASP ZAP                                     Kubernetes
                                           kubect1

                                INFRASTRUCTURE
                                Docker Desktop K8s
                                AWS EC2 / EKS
                                (Terraform IaC)
```

2. TP1 — Notes App : Premiers Pas DevSecOps

2.1 Présentation du Projet

Le premier TP constitue l'introduction au DevSecOps. **Notes App** est une application web simple permettant d'ajouter, consulter et supprimer des notes. Sa simplicité permet de se concentrer sur les fondamentaux : Dockerisation, tests automatisés, et mise en place d'une première pipeline CI/CD.

🔗 Stack Technique — TP1

Backend : Node.js + Express
Frontend : HTML/CSS/JS (statique)
Tests : Jest (5 tests unitaires)
CI/CD : GitHub Actions (solution alternative GitLab)
Container : Docker (node:18-alpine)

2.2 Architecture de l'Application

📁 Structure du Projet

```
tp1/  
  app.js           # Serveur Express (API REST + fichiers statiques)  
  notes.js         # Logique métier (CRUD en mémoire)  
  notes.test.js    # Tests Jest (5 cas de test)  
  public/  
    index.html     # Interface utilisateur  
  package.json  
  Dockerfile  
  .github/  
    workflows/  
      ci.yml       # Pipeline GitHub Actions
```

2.3 Dockerfile

```
1 FROM node:18-alpine  
2 WORKDIR /app  
3 COPY package*.json ./  
4 RUN npm install --production  
5 COPY . .  
6 EXPOSE 3000  
7 CMD ["node", "app.js"]
```

Listing 2.1: Dockerfile TP1 — Image node:18-alpine

2.4 Tests Automatisés — Jest

✓ Résultats des Tests Jest — 5/5 Passés

```
Test Suites: 1 passed, 1 total
Tests:      5 passed, 5 total
Temps d'exécution: 1.425 secondes

Should add a new note          (13ms)
Should not add an empty note   (32ms)
Should delete an existing note (16ms)
Should throw error (note not found) (3ms)
Should return all notes        (1ms)
```

2.5 Pipeline CI/CD — GitHub Actions

GitLab posant un problème de vérification de compte, la solution GitHub Actions a été adoptée.

```
1 name: Note App CI/CD Pipeline
2
3 on:
4   push:
5     branches: [ "main" ]
6   pull_request:
7
8 jobs:
9   build:
10    runs-on: ubuntu-latest
11    steps:
12      - name: Checkout repository
13        uses: actions/checkout@v3
14
15      - name: Setup Node.js
16        uses: actions/setup-node@v3
17        with:
18          node-version: 18
19
20      - name: Install dependencies
21        run: npm install
22
23      - name: Run tests
24        run: npm test
25
26      - name: Security audit
27        run: npm audit --audit-level=high
```

Listing 2.2: .github/workflows/ci.yml — Pipeline GitHub Actions

✓ Résultat Pipeline TP1

Pipeline #**GitHub Actions** — 5 étapes passées en **≈18 secondes** :
Checkout → Setup Node.js → Install → Tests → Security Audit

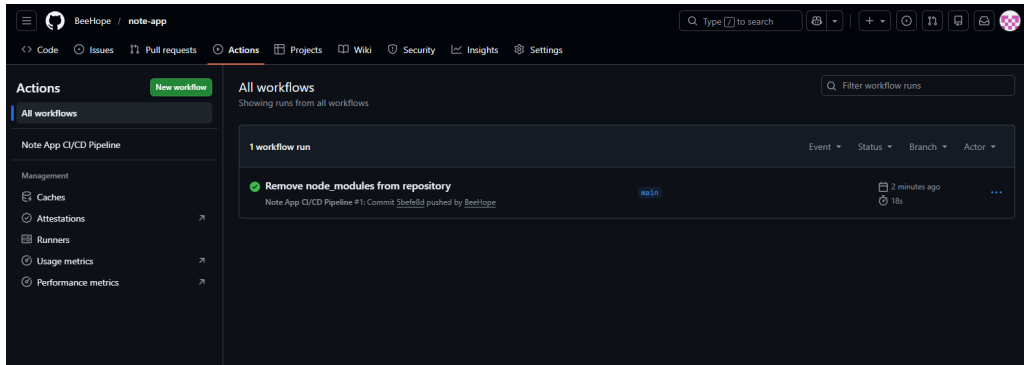


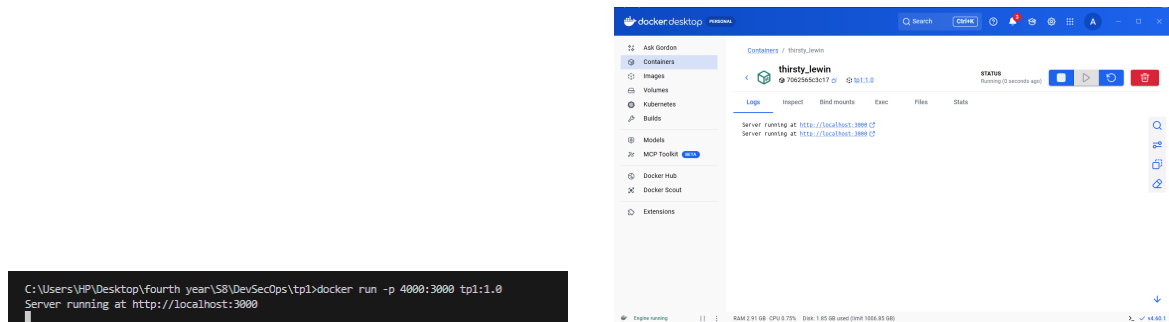
Figure 2.1: Pipeline GitHub Actions — Exécution réussie en ≈18 secondes

2.6 Résultats de l'Exécution Docker

```
# Build de l'image
docker build -t tp1:1.0 .

# Lancement du conteneur (port host 4000      conteneur 3000)
docker run -p 4000:3000 tp1:1.0
```

Listing 2.3: Commandes Docker TP1



(a) Lancement du conteneur

(b) Docker Desktop — Container Running

Figure 2.2: Exécution Docker TP1 — Notes App opérationnelle

3. TP2 — WeatherOps : Pipeline DevSecOps Complet

3.1 Présentation du Projet

WeatherOps est une API REST météo développée avec Node.js/Express, exposant des données simulées pour 5 villes (Paris, London, Tokyo, New York, Dubai). Ce projet introduit un pipeline GitLab CI/CD en 5 stages, avec déploiement Kubernetes et infrastructure AWS via Terraform.

Stack Technique — WeatherOps

Backend :	Node.js + Express
Tests :	Jest + Supertest (6 tests)
Sécurité :	Détection de secrets hardcodés (grep/regex)
CI/CD :	GitLab CI/CD — 5 stages
Container :	Docker (node:20-alpine)
Orchestration :	Kubernetes (namespace, deployment, service)
Cloud :	AWS EC2 via Terraform (t3.micro)

3.2 Architecture du Projet

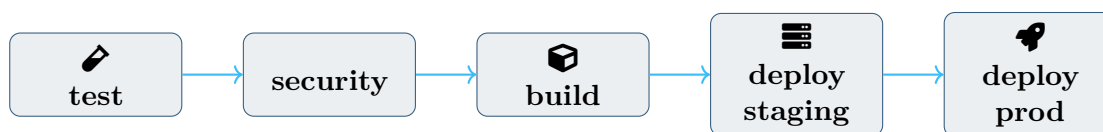
Structure WeatherOps

```
weatherops/  
  src/  
    app.js          # Express app (routes + logique)  
    server.js       # Point d'entrée HTTP  
  views/           # Templates HTML  
  tests/  
    app.test.js     # Tests Jest / Supertest  
  k8s/  
    namespace.yaml  
    deployment.yaml  
    service.yaml  
  Dockerfile  
  .gitlab-ci.yml  
  .env
```

3.3 Endpoints REST

Méthode	Route	Description
GET	/	Dashboard principal — liste les villes
GET	/health	Statut API (timestamp, status ok)
GET	/weather	Toutes les villes disponibles
GET	/weather/:city	Détails météo d'une ville

3.4 Pipeline GitLab CI/CD — 5 Stages



3.4.1 Stage 1 — Test (Jest/Supertest)

```

1 test:
2   stage: test
3   image: node:20-alpine
4   before_script:
5     - npm ci
6   script:
7     - npm test
  
```

Listing 3.1: Stage test — Installation et tests automatisés

✓ 6 Tests Passés

GET / retourne du HTML	GET /weather retourne du HTML
GET /health retourne du HTML	GET /weather/paris retourne du HTML
GET /weather/tokyo retourne HTML	GET /weather/unknown retourne 404

3.4.2 Stage 2 — Security (Détection de Secrets)

```

1 security:
2   stage: security
3   image: alpine:latest
4   script:
5     - |
6       if grep -rE 'API_KEY\s*=\s*["\x27]?sk-[a-zA-Z0-9-]+' \
7         --include="*.js" --include="*.json" . 2>/dev/null; then
8         echo "ERREUR : secret detecte"
9         exit 1
10      fi
11      echo "Aucun secret suspect detecte."
  
```

Listing 3.2: Stage security — Détection regex des clés API

⚠️ Vulnérabilité Détectée

La valeur `const API_KEY = "sk-demo-123456789"` dans `app.js` déclenche un **échec du pipeline**. Correction : utiliser `process.env.API_KEY` et injecter la valeur via les variables CI/CD GitLab.

```

Horodatage du journal en UTC.
22 12:43:49 $ apk add --no-cache grep
23 12:43:49 (1/2) Installing pcre2 (10.47-r0)
24 12:43:49 (2/2) Installing grep (3.12-r0)
25 12:43:49 Executing busybox-1.37.0-r30.trigger
26 12:43:49 OK: 9156 KiB in 18 packages
27 12:43:49 $ echo "🔍 Scanning for potential secrets..." # collapsed multi-line command
28 12:43:49 🔍 Scanning for potential secrets...
29 12:43:49 ./git/logs/refs/remotes/origin/main:0000000000000000000000000000000000000000 fbb15089baec172f7a3628c3
7e9be37dab77a9c root <root@runner-j2nyww-sk-project-79719627-concurrent-0.(none)> 1771937028 +0000
fetch origin --no-recuse-submodules +refs/pipelines/2346276181:refs/pipelines/2346276181 +refs/heads/m
ain:refs/remotes/origin/main --depth 20 --prune --quiet: storing head
30 12:43:49 ./git/logs/HEAD:0000000000000000000000000000000000000000 fbb15089baec172f7a3628c37e9be37dab77a9c root
<root@runner-j2nyww-sk-project-79719627-concurrent-0.(none)> 1771937028 +0000 checkout: moving from n
one to fbb15089baec172f7a3628c37e9be37dab77a9c
31 12:43:49 ./gitlab-ci.yml:# Si le code contient un motif type API_KEY=sk-demo-..., le job échoue
32 12:43:49 ./gitlab-ci.yml: if grep -R -E '(password|s*=[passw|s*=[api[_]?key|s*=[secret|s*=[token|s*=[sk-
|AKIA|BEGIN PRIVATE KEY)'; then
33 12:43:49 ./gitlab-ci.yml: - terraform apply -auto-approve -var "docker_image=$CI_REGISTRY_IMAGE" -var "docke
r_tag=$CI_COMMIT_SHORT_SHA" -var "registry_url=$CI_REGISTRY" -var "registry_user=$CI_REGISTRY_USER" -va
r "registry_password=$CI_REGISTRY_PASSWORD"
34 12:43:49 ./gitlab-ci.yml: - terraform apply -auto-approve -var "docker_image=$CI_REGISTRY_IMAGE" -var "docke
r_image_tag=latest" -var "registry_url=https://$CI_REGISTRY" -var "registry_user=$CI_REGISTRY_USER" -va
r "registry_password=$CI_REGISTRY_PASSWORD"
35 12:43:49 ./src/app.js:const apiKey="sk-demo-123456789";
36 12:43:49 ✖ Potential secret detected!
37 12:43:49 Cleaning up project directory and file based variables
38 12:43:50 ERROR: Job failed: exit code 1
    
```

Figure 3.1: Pipeline échoué — Secret hardcodé détecté (exit code 1)

3.4.3 Stage 3 — Build (Docker)

```

1 build:
2   stage: build
3   image: docker:24
4   services:
5     - docker:24-dind
6   variables:
7     DOCKER_TLS_CERTDIR: "/certs"
8   script:
9     - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD
      $CI_REGISTRY
10    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
      -t $CI_REGISTRY_IMAGE:latest .
11    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
12    - docker push $CI_REGISTRY_IMAGE:latest
    
```

Listing 3.3: Stage build — Build multi-tag et push vers GitLab Registry

3.4.4 Stages 4 & 5 — Deploy Staging et Production

Stratégie de Déploiement

Branche	Stage	Déclenchement
develop	deploy_staging	Automatique
main	deploy_production	Manuel (bouton GitLab)

Cette stratégie garantit qu’aucun déploiement non supervisé n’atteint la production.

3.5 Infrastructure AWS — Terraform

L’infrastructure de staging a été provisionnée avec Terraform : instance EC2 t3.micro (Amazon Linux), paire de clés SSH, et IP publique 54.211.68.238.

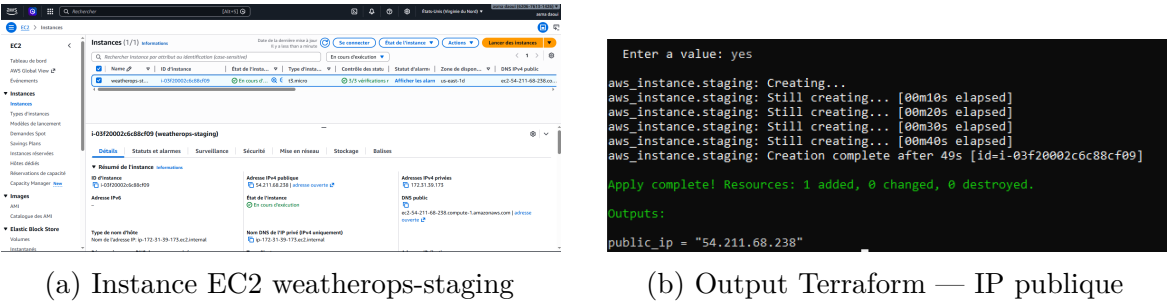


Figure 3.2: Infrastructure AWS WeatherOps provisionnée par Terraform

3.6 Pipeline Réussi

Après correction du secret hardcodé (`process.env.API_KEY`), le pipeline s’exécute avec succès.

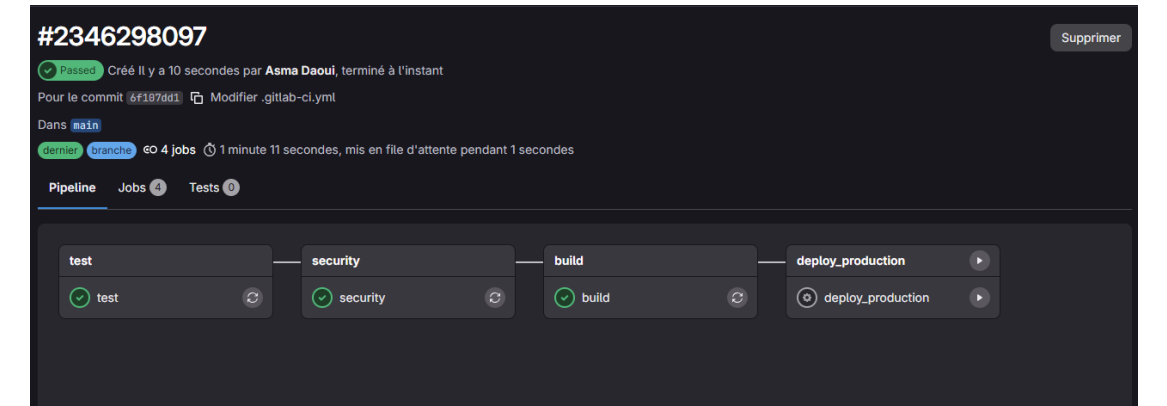


Figure 3.3: Pipeline WeatherOps — 4 jobs passés en 1min 11s sur la branche main

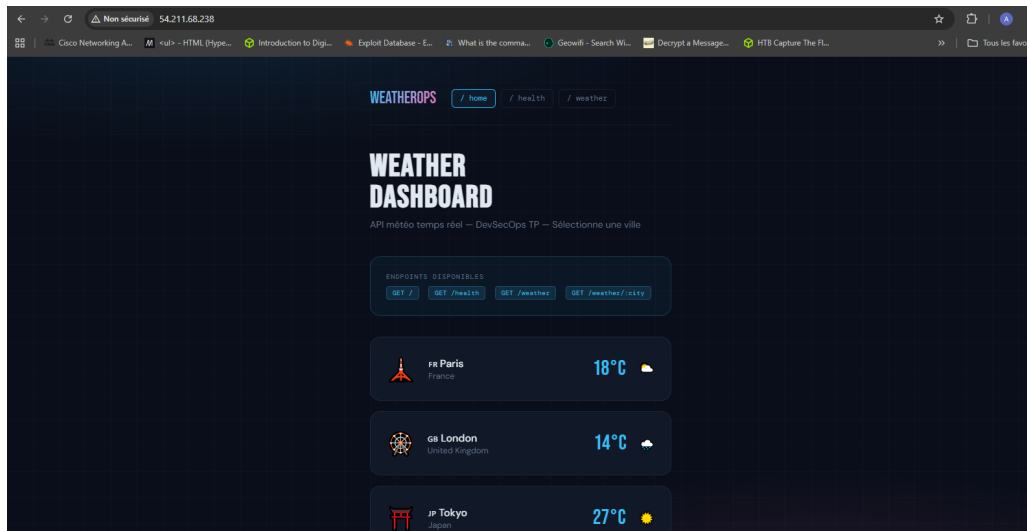


Figure 3.4: WeatherOps déployé et accessible via l'IP publique AWS

4. TP3 — SecNotes : SAST, DAST et Kubernetes

4.1 Présentation du Projet

SecNotes est une API REST Express/MongoDB conçue comme un *lab de sécurité*. Elle expose volontairement **10 vulnérabilités pédagogiques** pour permettre leur détection par les outils SAST et DAST. C'est le projet le plus complet en termes de sécurité.

Stack Technique — SecNotes

Backend :	Node.js + Express + MongoDB
Tests :	Jest + Supertest + MongoDB Memory Server
SAST :	SonarQube
DAST :	OWASP ZAP (baseline scan)
CI/CD :	GitLab CI/CD — 7 stages, 12 jobs
Container :	Docker (<code>node:20</code> , volontairement non sécurisé)
Orchestration :	Kubernetes local
IaC :	Terraform (cluster EKS AWS)

4.2 Vulnérabilités Pédagogiques

ID	Catégorie	Description
V1	NoSQL Injection	Requêtes Mongoose non filtrées
V2	Secrets en dur	JWT secret hardcodé dans le code
V3	Validation absente	Aucun schéma de validation des entrées
V4	IDOR	Contrôle d'accès insuffisant sur les notes
V5	Dépendances obsolètes	<code>express 4.17.1</code> , <code>jsonwebtoken 8.5.1</code> , <code>lodash 4.17.19</code>
V6	Hashage faible	Mots de passe hashés en MD5
V7	Dockerfile non sécurisé	Exécution en <code>root</code> , pas de multi-stage
V8	Logs verbeux	Informations sensibles dans les logs
V9	Secrets Kubernetes	Secret en base64 non chiffré
V10	CORS permissif	Toutes les origines autorisées (*)

4.3 Pipeline GitLab CI/CD — 7 Stages, 12 Jobs

Stage	Job	Rôle
install	install_dependencies	npm ci — cache node_modules
test	run_tests	Jest + Supertest + MongoDB Memory Server
sast	sonarqube_scan	Analyse statique SonarQube
build	docker_build	Build + tag image Docker
deploy_local	deploy_k8s_local	kubectl apply Kubernetes local
zap	zap_dast_scan	OWASP ZAP baseline scan
terraform_plan	terraform_plan	terraform plan EKS (manuel)

4.3.1 Stage SAST — SonarQube

```

1 sonar.projectKey=secnotes-vuln-app
2 sonar.projectName=SecNotes Vulnerable App
3 sonar.projectVersion=1.0
4 sonar.sources=src
5 sonar.tests=tests
6 sonar.sourceEncoding=UTF-8
7 sonar.javascript.lcov.reportPaths=coverage/lcov.info

```

Listing 4.1: sonar-project.properties — Configuration SonarQube

⚠ Résultats SonarQube

SonarQube a détecté les vulnérabilités **V1 à V6** du tableau : injection NoSQL, secrets hardcodés (JWT), absence de validation, IDOR, dépendances obsolètes, hashage MD5. Ces résultats constituent la base d'un rapport de remédiation.

4.3.2 Stage DAST — OWASP ZAP

```

1 zap_dast_scan:
2   stage: zap
3   only:
4     - main
5   script:
6     - docker run --rm -v "${zapReport}:/zap/wrk" \
7       ghcr.io/zaproxy/zaproxy:stable \
8       zap-baseline.py -t $APP_URL -r zap-report.html -I
9   artifacts:
10    paths:
11      - zap-report/
12    when: always
13    allow_failure: true

```

Listing 4.2: Stage zap — Baseline DAST Scan

ℹ Résultats OWASP ZAP

Le scan DAST a identifié **2 alertes Medium** sur l'environnement local :

- En-têtes de sécurité HTTP manquants (CSP, X-Frame-Options)
- CORS permissif (V10) — toutes les origines acceptées

Le stage est configuré avec `allow_failure: true` pour ne pas bloquer le pipeline sur les vulnérabilités intentionnelles.

4.3.3 Infrastructure as Code — Terraform EKS AWS


```

1 resource "aws_eks_cluster" "secnotes" {
2   name      = var.cluster_name
3   role_arn  = aws_iam_role.eks_cluster_role.arn
4   version   = "1.29"
5   vpc_config {
6     subnet_ids = [aws_subnet.public_a.id, aws_subnet.public_b.id]
7   }
8 }
9
10 resource "aws_eks_node_group" "secnotes_nodes" {
11   cluster_name      = aws_eks_cluster.secnotes.name
12   node_group_name   = "${var.project_name}-nodes"
13   node_role_arn     = aws_iam_role.eks_node_role.arn
14   subnet_ids        = [aws_subnet.public_a.id,
15     aws_subnet.public_b.id]
16   instance_types    = ["t3.micro"]
17   scaling_config {
18     desired_size = 1
19     max_size     = 2
20     min_size     = 1
21   }
22 }

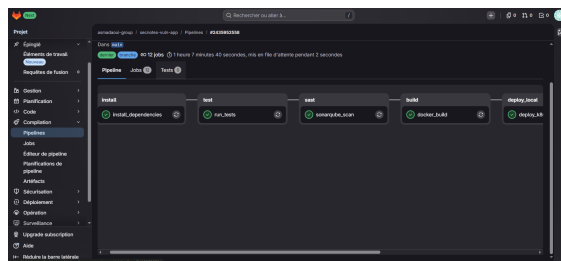
```

Listing 4.3: Cluster EKS SecNotes — main.tf (extrait)

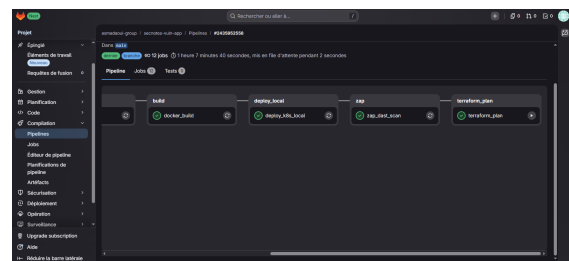
4.4 Résultats — Pipeline #2435952558

✓ Pipeline Complet en 1h07m40s

12 jobs exécutés avec succès — les stages `zap` et `sonarqube_scan` ont `allow_failure: true` pour ne pas bloquer le pipeline sur les vulnérabilités pédagogiques intentionnelles.



(a) Stages install, test, sast, build



(b) Stages deploy, zap, terraform_plan

Figure 4.1: Pipeline SecNotes #2435952558 — tous les 12 jobs en succès

5. TP4 — CRUD App : Trivy et Kubernetes Multi-Replicas

5.1 Présentation du Projet

CRUD App est une API REST Node.js/MongoDB avec 5 routes (Create, Read, Update, Delete, Health). Ce projet introduit le scan d'image **Trivy**, le multi-stage Dockerfile, et le déploiement Kubernetes avec **3 replicas** et un StatefulSet MongoDB.

Stack Technique — CRUD App

Backend :	Node.js + Express + MongoDB
Tests :	Jest + Supertest
Image Scan :	Trivy (HIGH/CRITICAL)
CI/CD :	GitLab CI/CD — 4 stages
Container :	Docker multi-stage build (node:20-alpine)
Orchestration :	Docker Desktop Kubernetes (3 replicas + StatefulSet MongoDB)
Cloud :	AWS EC2 via Terraform

5.2 Dockerfile Multi-Stage

```
1 # Stage 1: Build
2 FROM node:20-alpine AS builder
3 WORKDIR /app
4 COPY package*.json ./
5 RUN npm ci --only=production
6
7 # Stage 2: Runtime (image minimale)
8 FROM node:20-alpine
9 WORKDIR /app
10 COPY --from=builder /app/node_modules ./node_modules
11 COPY src/ ./src/
12 COPY package.json ./
13
14 EXPOSE 3000
15
16 HEALTHCHECK --interval=30s --timeout=5s \
17   CMD wget -q0- http://localhost:3000/health || exit 1
18
19 CMD ["node", "src/index.js"]
```

Listing 5.1: Dockerfile CRUD App — Multi-stage build sécurisé

💡 Avantages du Multi-Stage Build

- Image finale ne contient **pas les outils de build** (npm, compilateurs)
- Surface d'attaque réduite — uniquement le runtime nécessaire
- Image plus légère → transferts plus rapides
- HEALTHCHECK intégré pour Kubernetes

5.3 Pipeline GitLab CI/CD — 4 Stages

```

1 stages:
2   - build
3   - test
4   - security
5   - deploy
6
7 # Stage security      Scan Trivy
8 security:
9   stage: security
10  image: docker:24
11  services:
12    - docker:24-dind
13  variables:
14    DOCKER_TLS_CERTDIR: "/certs"
15    TRIVY_AUTH_URL: $CI_REGISTRY
16    TRIVY_USERNAME: $CI_REGISTRY_USER
17    TRIVY_PASSWORD: $CI_REGISTRY_PASSWORD
18  before_script:
19    - apk add --no-cache curl
20    - curl -sL
21      https://raw.githubusercontent.com/aquasecurity/trivy/main/
22      contrib/install.sh | sh -s -- -b /usr/local/bin
23  script:
24    - trivy image --exit-code 0 --severity HIGH,CRITICAL
25      $CI_REGISTRY_IMAGE:latest
26  only:
27    - main
  
```

Listing 5.2: .gitlab-ci.yml CRUD App — Stage security avec Trivy

Trivy — Scan d'Image Docker

Trivy analyse l'image Docker à la recherche de vulnérabilités connues (CVE) dans les packages OS et les dépendances Node.js. Le flag `-severity HIGH,CRITICAL` filtre uniquement les vulnérabilités sérieuses. Le flag `-exit-code 0` permet au pipeline de continuer même en présence de vulnérabilités (mode informatif).

5.4 Kubernetes — 3 Replicas + StatefulSet MongoDB

```

1 spec:
2   replicas: 3
3   template:
4     spec:
5       containers:
6         - name: crud-app
7           image:
8             registry.gitlab.com/asmadaoui-group/crud-app:latest
  
```

```

8     ports:
9       - containerPort: 3000
10    readinessProbe:
11      httpGet:
12        path: /health
13        port: 3000
14        initialDelaySeconds: 5
15        periodSeconds: 10
16    livenessProbe:
17      httpGet:
18        path: /health
19        port: 3000
20        initialDelaySeconds: 15
21        periodSeconds: 20

```

Listing 5.3: deployment.yaml CRUD App — 3 replicas avec probes

✓ Pipeline CRUD App Réussi

4 stages exécutés avec succès :

build → test → security (Trivy) → deploy (kubectl)

Application accessible sur <http://localhost:30080>

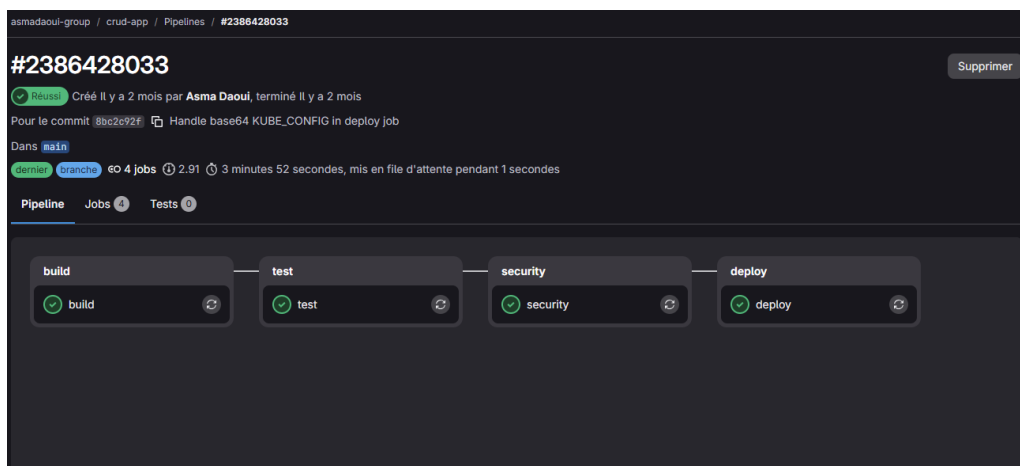


Figure 5.1: Pipeline CRUD App — 4 stages : build, test, security, deploy

6. TP5 — AWS Terraform : Infrastructure as Code

6.1 Présentation du Projet

Ce TP se concentre sur l'aspect **Infrastructure as Code** (IaC) avec Terraform sur AWS. L'objectif est de provisionner automatiquement une instance EC2 et un bucket S3, puis de déployer l'application Note App via une pipeline GitLab CI/CD complète.

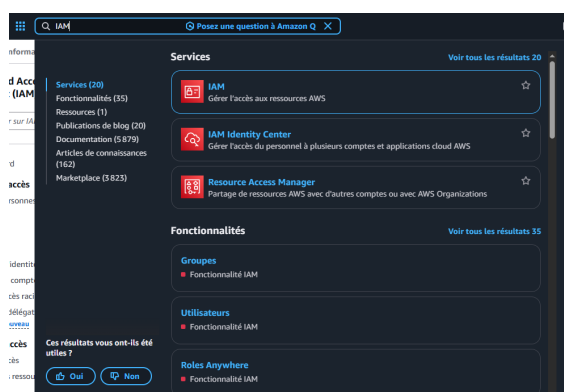
aws Stack Technique — AWS Terraform

IaC :	Terraform (provider AWS ~5.0)
Cloud :	AWS EC2 (t3.micro, Amazon Linux 2023) + S3
Accès :	IAM User + Access Key + paire de clés SSH
CI/CD :	GitLab CI/CD (build, test, security, package, deploy)
Registry :	GitLab Container Registry
Env. Local :	WSL Ubuntu sur Windows

6.2 Configuration IAM AWS

⚠ Bonnes Pratiques IAM

Pour ce lab : AdministratorAccess est attaché à l'utilisateur IAM Terraform pour simplifier les opérations. **En production :** utiliser le principe du moindre privilège — attacher uniquement les politiques strictement nécessaires (AmazonEC2FullAccess, AmazonS3FullAccess, etc.).



(a) Création de l'utilisateur IAM



(b) Création de la clé d'accès CLI

Figure 6.1: Configuration IAM AWS pour Terraform

6.3 Configuration Terraform

```

1 provider "aws" {
2     region = "us-east-1"
3 }
4
5 resource "aws_s3_bucket" "my_bucket" {
6     bucket = "asma-terraform-bucket-2026-unique"
7     tags   = { Name = "TerraformBucket", Environment = "Dev" }
8 }
9
10 resource "aws_security_group" "allow_ssh" {
11     name = "allow_ssh_terraform"
12     ingress {
13         from_port = 22; to_port = 22; protocol = "tcp"
14         cidr_blocks = ["0.0.0.0/0"]
15     }
16     egress {
17         from_port = 0; to_port = 0; protocol = "-1"
18         cidr_blocks = ["0.0.0.0/0"]
19     }
20 }
21
22 resource "aws_instance" "my_instance" {
23     ami           = "ami-0f3caa1cf4417e51b"
24     instance_type = "t3.micro"
25     key_name      = "terraformkeyinstance"
26     vpc_security_group_ids = [aws_security_group.allow_ssh.id]
27     tags = { Name = "TerraformEC2" }
28 }
29
30 output "instance_public_ip" {
31     value = aws_instance.my_instance.public_ip
32 }

```

Listing 6.1: main.tf — EC2 + S3 + Security Group

```

terraform init    # Initialiser providers et modules
terraform plan    # Pr visualiser les changements
terraform apply   # Provisionner l'infrastructure
terraform output  # Afficher l'IP publique de l'instance

```

Listing 6.2: Commandes Terraform — Déploiement

6.4 Pipeline GitLab CI/CD — Stages Package et Deploy

```

1 package-docker:
2     stage: package
3     image: docker:latest
4     services:
5         - docker:dind
6     variables:
7         DOCKER_TLS_CERTDIR: ""
8     script:
9         - echo "$CI_JOB_TOKEN" | docker login -u "$CI_REGISTRY_USER"
10         --password-stdin $CI_REGISTRY
11         - docker build -t $CI_REGISTRY_IMAGE:$APP_VERSION .
12         - docker tag $CI_REGISTRY_IMAGE:$APP_VERSION
          $CI_REGISTRY_IMAGE:latest

```

```

13 - docker push $CI_REGISTRY_IMAGE:$APP_VERSION
14 - docker push $CI_REGISTRY_IMAGE:latest
15 only:
16 - main

```

Listing 6.3: Stage package — Build et push image Docker vers GitLab Registry

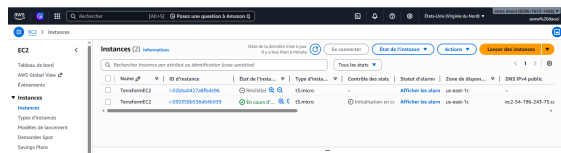
```

1 deploy-staging:
2   stage: deploy
3   image: docker:latest
4   services:
5     - docker:dind
6   script:
7     - echo "$CI_JOB_TOKEN" | docker login -u "$CI_REGISTRY_USER"
8       --password-stdin $CI_REGISTRY
9     - docker pull $CI_REGISTRY_IMAGE:$APP_VERSION
10    - docker stop tp1-staging || true
11    - docker rm tp1-staging || true
12    - docker run -d --name tp1-staging -p 8000:3000
13      $CI_REGISTRY_IMAGE:$APP_VERSION
14   environment:
15     name: staging
16     url: http://staging.local:8000
17   only:
18     - main

```

Listing 6.4: Stage deploy — Pull et run sur l'instance EC2

6.5 Résultats



(a) Instance TerraformEC2 dans la console AWS

```

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_instance.my_instance: Destroying... [id=i-02bba0427a8fbde96]
aws_instance.my_instance: Still destroying... [id=i-02bba0427a8fbde96, 00m10s elapsed]
aws_instance.my_instance: Still destroying... [id=i-02bba0427a8fbde96, 00m20s elapsed]
aws_instance.my_instance: Still destroying... [id=i-02bba0427a8fbde96, 00m30s elapsed]
aws_instance.my_instance: Destruction complete after 31s
aws_instance.my_instance: Creating...
aws_instance.my_instance: Still creating... [00m10s elapsed]
aws_instance.my_instance: Creation complete after 15s [id=i-030358b538d64b939]

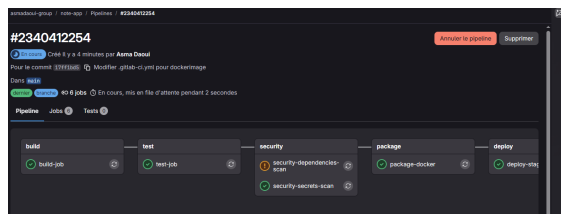
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:
instance_public_ip = "54.196.243.75"
ubuntu@DESKTOP-DJVB8EK:~/terraform-lab$

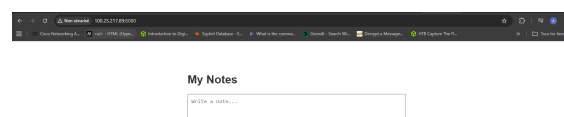
```

(b) Output Terraform — IP publique 54.196.243.75

Figure 6.2: Infrastructure AWS provisionnée par Terraform avec succès



(a) Pipeline GitLab — tous les stages en succès



(b) Note App accessible en live sur EC2

Figure 6.3: Déploiement complet — Pipeline CI/CD + Application sur AWS EC2

7. Analyse de Sécurité Transversale

7.1 Outils de Sécurité Utilisés

Outil	Type	Projet	Rôle
grep/regex	SAST basique	WeatherOps	Détection de patterns de secrets (API keys)
SonarQube	SAST complet	SecNotes	Analyse statique du code, détection de vulnérabilités
Trivy	Image Scan	CRUD App	Scan CVE des images Docker (OS + dépendances)
OWASP ZAP	DAST	SecNotes	Tests dynamiques sur l'application en cours d'exécution
npm audit	Dépendances	TP1, tous	Audit des vulnérabilités dans les packages npm

7.2 Matrice des Vulnérabilités par Projet

Vulnérabilité	TP1	WeatherOps	SecNotes	CRUD App
Secret hardcodé	—	⚠	⚠	—
Dépendances obsolètes	—	—	⚠	—
Docker root user	—	—	⚠	✔
CORS permissif	—	—	⚠	—
Secrets K8s base64	—	—	⚠	—
.env dans le repo	⚠	⚠	⚠	⚠
NoSQL Injection	—	—	⚠	—
Hashage MD5	—	—	⚠	—

⚠ = Vulnérabilité présente ✔ = Bonne pratique appliquée — = Non applicable

7.3 Bonnes Pratiques Appliquées

- **.gitignore / .dockerignore** — exclure `node_modules`, `.env`, fichiers de test de l'image et du dépôt dans tous les projets.
- **npm ci** — installation reproductible et déterministe (vs `npm install`) dans tous les pipelines.
- **Images Alpine** (`node:20-alpine`) — surface d'attaque minimale, images légères.
- **Multi-stage Docker build** (CRUD App) — séparation build/runtime.

- **Probes Kubernetes** (CRUD App) — `readinessProbe` et `livenessProbe` sur `/health`.
- **Variables CI/CD GitLab** — secrets injectés via l'interface GitLab (protégées, masquées).
- **Stratégie staging/production** (WeatherOps) — déploiement automatique en staging, manuel en production.

7.4 Améliorations Recommandées

💡 Roadmap Sécurité — Prochaines Étapes

1. **HashiCorp Vault / AWS Secrets Manager** — Gestion centralisée des secrets (remplace les variables CI/CD et les fichiers `.env`).
2. **Sealed Secrets Kubernetes** — Chiffrement réel des secrets K8s (remplace le base64 non chiffré).
3. **NetworkPolicy Kubernetes** — Isolation réseau des pods pour limiter les mouvements latéraux.
4. **OWASP ZAP Full Scan** — Scan DAST complet (vs baseline) sur l'environnement staging.
5. **Dependabot / Renovate** — Mise à jour automatique des dépendances vulnérables.
6. **SBOM (Software Bill of Materials)** — Inventaire des composants pour la traçabilité.

8. Comparaison des Pipelines et Outils

8.1 Tableau Comparatif des Pipelines

Feature	TP1	Weather	SecNotes	CRUD	Terraform
CI/CD Platform	GitHub	GitLab	GitLab	GitLab	GitLab
Stages count	3	5	7	4	5
Tests auto.	✓	✓	✓	✓	✓
Secret detect.	–	✓	✓	–	–
SAST	–	–	SonarQube	–	–
Image scan	–	–	–	Trivy	–
DAST	–	–	OWASP ZAP	–	–
Kubernetes	–	✓	✓	✓	–
AWS	–	EC2	EKS (plan)	EC2	EC2 + S3
Terraform	–	✓	✓	✓	✓
Registry	–	GitLab	–	GitLab	GitLab

8.2 GitLab CI/CD vs GitHub Actions

Caractéristique	GitLab CI/CD	GitHub Actions
Fichier de config	<code>.gitlab-ci.yml</code>	<code>.github/workflows/*.yml</code>
Syntaxe	YAML (stages/jobs)	YAML (jobs/steps)
Runner	GitLab Runner (self-hosted / SaaS)	GitHub-hosted runner
Container Registry	Intégré (natif)	GitHub Packages
Kubernetes	kubectl natif	via actions tierces
Secrets	Variables CI/CD	GitHub Secrets
Temps d'exécution	Variable	≈18s (TP1)
Gratuit	Jusqu'à 400 min/mois	Illimité (repos publics)

9. Conclusion et Bilan

9.1 Acquis du Module DevSecOps

Ce parcours de 5 projets a permis de construire, étape par étape, une maîtrise complète de la chaîne DevSecOps :

- **Développement sécurisé** — Écriture de code avec conscience des vulnérabilités (secrets, injections, CORS), tests automatisés systématiques.
- **Intégration continue** — Configuration de pipelines GitLab CI/CD et GitHub Actions avec stages progressifs (install, test, security, build, deploy).
- **Shift Left Security** — Intégration de la sécurité dès le commit : détection de secrets par regex, SAST SonarQube, scan d'image Trivy, DAST OWASP ZAP.
- **Conteneurisation** — Maîtrise de Docker (Dockerfile simple puis multi-stage), Docker Compose, gestion du registry GitLab.
- **Orchestration Kubernetes** — Déploiement sur Docker Desktop K8s et Kubernetes AWS, namespace, deployment, service NodePort, ConfigMap, Secret, StatefulSet, probes.
- **Infrastructure as Code** — Terraform pour provisionner EC2, S3, VPC, clusters EKS sur AWS. Reproductibilité et versionning de l'infrastructure.
- **Gestion des environnements** — Distinction staging (automatique) / production (manuel), variables CI/CD, kubeconfig encodé.

9.2 Bilan des Livrables

✓ Récapitulatif des Livrables

- 📁 5 applications Node.js (Notes App, WeatherOps, SecNotes, CRUD App, TP AWS)
- 🔧 Tests automatisés (Jest + Supertest) — 100% pass
- 🔍 4 types d'analyse de sécurité (regex, SAST, image scan, DAST)
- 📦 Images Docker multi-tag sur GitLab Container Registry
- 🏠 Déploiements Kubernetes (local + staging + production)
- ☁️ Infrastructure AWS provisionnée par Terraform (EC2, S3, EKS plan)
- 🔗 Pipelines CI/CD GitLab (jusqu'à 7 stages, 12 jobs)
- 🔄 Pipeline GitHub Actions (solution de repli opérationnelle)

“ Philosophie DevSecOps Retenue

“La sécurité n’est pas une fonctionnalité à ajouter à la fin : c’est une discipline à intégrer à chaque commit, chaque build, chaque déploiement. Le pipeline CI/CD est le gardien automatisé qui veille à ce que rien de non sécurisé ne passe en production.”

Le DevSecOps transforme la sécurité d’une contrainte perçue comme un frein en un **accélérateur de confiance** : chaque badge vert du pipeline est la preuve que le code est testé, scanné, et prêt pour la production.

A. Variables CI/CD GitLab — Référence Complète

Variable	Projet	Description
CI_REGISTRY_USER	Tous	Username pour le registry Docker GitLab
CI_REGISTRY_PASSWORD	Tous	Token ou mot de passe registry
CI_REGISTRY	Tous	URL du registry (registry.gitlab.com)
CI_REGISTRY_IMAGE	Tous	Chemin complet de l'image
KUBE_CONFIG	WeatherOps, CRUD	Kubeconfig encodé en base64
api_key	WeatherOps	Clé API (protégée, masquée, cachée)
AWS_ACCESS_KEY_ID	Terraform	Clé d'accès AWS
AWS_SECRET_ACCESS_KEY	Terraform	Clé secrète AWS

B. Données Météo — WeatherOps

Ville	Pays	Condition	Temp.	Hum.	Vent
Paris	France	Partly Cloudy	18°C	65%	12 km/h
London	United Kingdom	Rainy	14°C	80%	20 km/h
Tokyo	Japan	Sunny	27°C	55%	8 km/h
New York	USA	Clear	22°C	50%	15 km/h
Dubai	UAE	Hot & Sunny	40°C	30%	5 km/h

C. Commandes de Référence

Docker

```
docker build -t image:tag .
docker run -p host:container image:tag
docker ps
docker logs <container>
docker login registry.gitlab.com
```

Kubernetes

```
kubectl apply -f k8s/
kubectl get pods -n <namespace>
kubectl rollout status deployment/<name> -n <namespace>
kubectl logs -l app=<name> -n <namespace>
kubectl port-forward svc/<service> 8080:80 -n <namespace>
```

Terraform

```
terraform init
terraform plan
terraform apply
terraform output
terraform destroy
```